STEP 1 — Create Notebook

Step 1:Import Required Libraries

```
# Natural Language Toolkit for text processing and tokenization
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords

# For counting n-grams efficiently
from collections import Counter, defaultdict

# Numerical computations
import numpy as np

# Optional: data handling
import pandas as pd

# Download required NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

Step 2:Load Dataset

```
# Sample corpus (≈ 1500+ words recommended; replace with file if needed)
# The file 'corpus.txt' was not found.
# For demonstration purposes, creating a sample text directly.
text = "This is a sample text. It contains multiple sentences for natural language processing tasks. We will use this text to d
# If you have 'corpus.txt', uncomment the lines below and ensure the file is uploaded to your Colab environment:
# with open("corpus.txt", "r", encoding="utf-8") as f:
#     text = f.read()

# Display sample text
print(text[:500])
```

```
This is a sample text. It contains multiple sentences for natural language processing tasks. We will use this text to demonstrat
```

Step 3:Preprocess Text

```
nltk.download('punkt_tab')

stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    text = text.lower()
    sentences = sent_tokenize(text)

    processed_sentences = []
    for sent in sentences:
        words = word_tokenize(sent)
        words = [w for w in words if w.isalpha()]
        words = [w for w in words if w not in stop_words]
        processed_sentences.append(['<s>'] + words + ['</s>'])

    return processed_sentences

sentences = preprocess_text(text)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Step 4:Build N-Gram Models

```
def build_ngram_model(sentences, n):
    ngrams = []
    for sent in sentences:
        ngrams.extend(list(nltk.ngrams(sent, n)))
    return Counter(ngrams)

unigram = build_ngram_model(sentences, 1)
bigram = build_ngram_model(sentences, 2)
trigram = build_ngram_model(sentences, 3)

vocab_size = len(unigram)
```

Step 5:Apply Add-One (Laplace) Smoothing

```
def laplace_prob(ngram, ngram_counts, prev_counts):
    return (ngram_counts[ngram] + 1) / (prev_counts + vocab_size)
```

Step 6:Sentence Probability Calculation

```
def sentence_probability(sentence, n, ngram_counts, prev_ngram_counts):
    tokens = ['<s>'] + word_tokenize(sentence.lower()) + ['</s>']
    tokens = [w for w in tokens if w.isalpha() or w in ['<s>', '</s>']]

    prob = 1.0
    for i in range(len(tokens) - n + 1):
        ngram = tuple(tokens[i:i+n])
        prev = tuple(tokens[i:i+n-1])

        prev_count = prev_ngram_counts[prev] if n > 1 else sum(unigram.values())
        prob *= laplace_prob(ngram, ngram_counts, prev_count)

    return prob

test_sentences = [
    "language models are useful",
    "this is a simple example",
    "natural language processing is fun",
    "unseen words reduce probability",
    "probability estimation is important"
]
```

Start coding or generate with AI.